COMP90024: Cluster and Cloud Computing

Assignment 2 Report

Cluster Cloud Computing A2 — Team 60

AutoPolis: Real-Time Social Media Sentiment Analysis for Australian Election Discourse

Team Members

Angqi Meng	1268867
Yichen Long	1497321
Xuan Wu	1483104
Zining Zhang	1508501
Jingqiu Meng	1506602

May 2025

Contents

1 Introduction

1.1 Motivation

The increasingly digital nature of political communication has made social media platforms a key frontier for public discourse and sentiment expression. In the Australian context, platforms like Mastodon, Reddit, and BlueSky are steadily shaping political narratives—particularly during national election periods. These platforms often feature unfiltered commentary around prominent political parties such as Labor, the Coalition, and the Greens, as well as figures like Peter Dutton.

Despite their analytical value, social media platforms present serious challenges for structured analysis. Their decentralised and fragmented nature, combined with real-time and high-volume content flows, complicates the systematic collection, integration, and interpretation of relevant data. These conditions hinder the ability to generate timely and actionable insights from digital political discourse.

This project is driven by the need for infrastructure that can bridge this analytical gap. Specifically, we aim to develop a resilient, modular, and dynamically scalable cloud-based data pipeline that supports automated harvesting, processing, enrichment, and analysis of political discussions across multiple platforms. The goal is not just to enable real-time monitoring, but also to support longitudinal research into evolving patterns of public engagement and electoral sentiment in Australia.

1.2 Objectives

The overarching goal of this system is to deliver a reusable and cloud-native data infrastructure that supports fine-grained political analysis at scale. The design and implementation are grounded in real-time stream processing principles, underpinned by container orchestration and fault-tolerant service isolation.

The system aims to:

1. Harvest public social media posts from Mastodon, Reddit, and BlueSky related to Australian federal elections, using targeted keyword filters and API-based streaming.
2. Enrich and annotate harvested posts with structured metadata, including inferred geolocation and topic classification (e.g., Labor, Coalition, Greens).
3. Index enriched data into ElasticSearch to enable efficient querying, aggregation, and downstream analytics.
4. Use Fission and Redis-backed queueing to achieve scalable, fault-isolated processing pipelines triggered asynchronously.
5. Expose the structured post data through a RestFUL API-based Falsk interface for use in notebooks.
6. Support scenario-specific insights such as sentiment trend detection, temporal spikes in campaign mentions, and regional engagement heatmaps.

This end-to-end platform blueprint serves as a foundation for computational political analysis—both in real-time and for retrospective studies—with a focus on reproducibility, scalability, and modularity.

1.3 Application Scenarios

This project is motivated by the growing need to understand the dynamics of political discourse across decentralised and evolving social media environments. Our system is designed to serve as an analytical backbone for researchers, policy analysts, and civic institutions who seek to monitor, compare, and interpret political narratives in real time.

Specifically, we focus on the Australian federal election context, where public sentiment and political attention are increasingly expressed via platforms such as Mastodon, Reddit, and BlueSky. Our platform is tailored to support the following high-level analytical scenarios:

1. **Party sentiment tracking:** Measuring political sentiment across parties (e.g. Labor, Coalition, Greens) by monitoring relevant hashtags, mentions, and engagement trends over time.
2. **Temporal analysis:** Identifying campaign bursts, peaks in attention, and shifts in discourse surrounding key political events such as debates, policy announcements, or controversies.
3. **Geospatial analysis:** Mapping region-specific political engagement to reveal hotspots of discussion or regional divides in sentiment.
4. **Cross-platform narrative analysis:** Understanding how different platforms shape discourse differently and detecting biases or topic fragmentation across sources.

These scenarios are not exhaustive but illustrate the breadth of political insight that can be derived from a platform capable of structured harvesting, annotation, and indexing of social media posts at scale. Our work thus aims to support both exploratory research and targeted investigative analysis of electoral sentiment and public opinion in Australia.

2 System Architecture

2.1 Overview of System Design

Our system adopts a modular, event-driven architecture tailored to the demands of high-throughput, real-time election discourse analytics. It is engineered to dynamically harvest, process, and analyse social media data from multiple platforms, while offering resilience, scalability, and fault tolerance through a containerised, cloud-native infrastructure. The structure is shown in Figure 1.

At a high level, the architecture is composed of the following logical stages: harvesting raw posts from multiple platforms, buffering data into a queue system, enriching content via stateless processors, indexing structured records into ElasticSearch, and exposing data for consumption via RestFUL API endpoints and Kibana dashboards. Each stage

operates independently and is containerised within a Kubernetes-managed environment on the NeCTAR Research Cloud.

The decision to structure the system this way stems from the need to support diverse workloads — including bursty traffic, asynchronous task execution, and parallelised content filtering. Kubernetes enables horizontal scaling of harvesters and processors, while Fission ensures lightweight and reactive function execution. This allows each processing layer to be elastic and fault-isolated.

To support robust system monitoring and debugging, each stage also emits logs and metrics, enabling a full observability stack across the pipeline. This design guarantees responsiveness even under volatile API latency or transient harvesting failures.

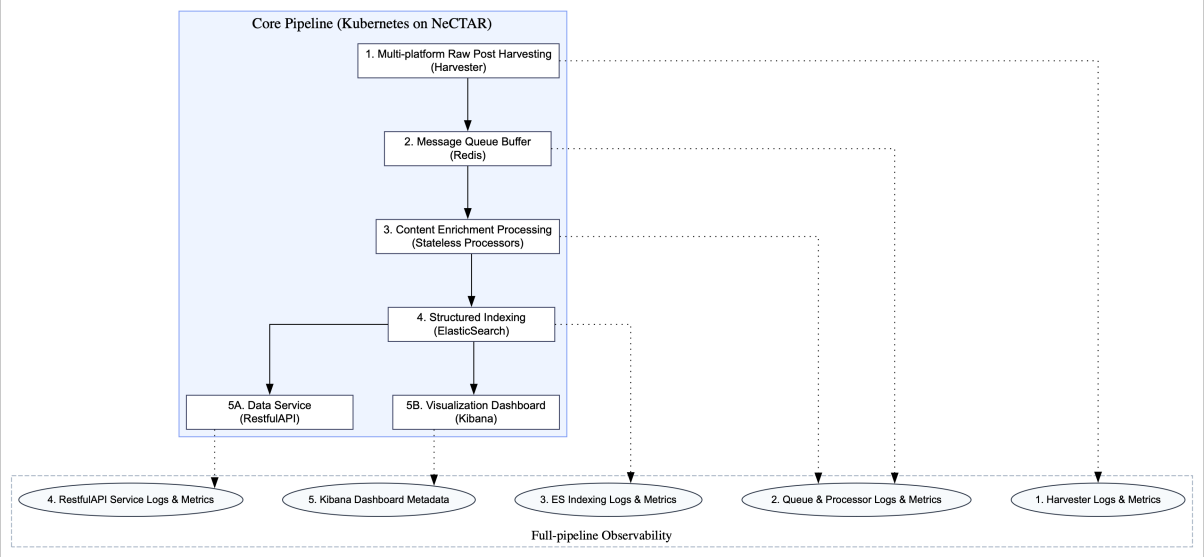

Figure 1: Overview diagram of the system

2.2 Melbourne Research Cloud

The system is deployed on the NeCTAR Research Cloud, leveraging its OpenStack-based virtual infrastructure to enable scalable and reproducible experimentation. We provisioned a dedicated Kubernetes cluster via the OpenStack dashboard, using Helm charts to manage service configuration, deployment, and scaling. The cloud system architecture is shown in Figure 2.

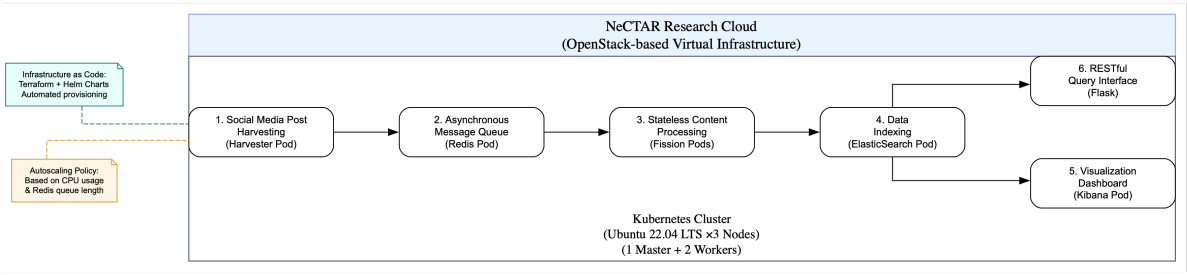

Figure 2: Cloud System Architecture

The cluster consists of one master node and four worker nodes, each running Ubuntu 22.04 (LTS) with 2 vCPUs, 9GB RAM, and 30GB disk storage. Public ingress for Kibana is enabled via a NodePort service, while all internal communication occurs over ClusterIP. Terraform scripts were used to automate provisioning and secret management.

```
(ccc) (base) devin@Devin-Mengs-MacBook-Air comp90024_team_60 % kubectl get pods -n fission
NAME                              READY   STATUS    RESTARTS      AGE
buildermgr-786556886-djp4h        1/1     Running   0             36d
executor-76c4c5b8f6-fccnf         1/1     Running   4 (15h ago)   36d
kubewatcher-6f475cfc6-s92km       1/1     Running   0             36d
mqtrigger-keda-5b895b7998-brjjw   1/1     Running   0             36d
router-5dc776db57-xnwbv           1/1     Running   0             36d
storagesvc-78f84f7c4d-f4mmg       1/1     Running   0             36d
timer-766994f4fd-xg5ms            1/1     Running   0             36d
webhook-7c86b98f-c6jdz            1/1     Running   0             36d
```

Figure 3: Pods Usage Detail

Container orchestration via Kubernetes allows for clear separation of roles across system components. Each pod is automatically set to play different roles. The figure above shows a clear overview of pods allocated.

Resource allocation is finely tuned — autoscaling policies dynamically adjust the number of active pods based on CPU usage and Redis queue length. This ensures the system gracefully absorbs load spikes without over-provisioning. By isolating components in their respective pods and namespaces, we also simplify debugging and updates, which is crucial in a modular setup like ours.

2.3 Data Collection

Our system is designed to capture and process public posts from the three major social media platforms, Bluesky, Reddit and Mastodon, in real time to support the collection of election-related posts and subsequent content analysis and visualization. On these three platforms, we manage variable parameters through ConfigMap. And the deployment is completed using the YAML file, making the update and maintenance of the function more convenient. Regarding the Harvester, we adopted the Time trigger approach and used the Cursor to record the crawled position, which was saved under the topic corresponding

to the query source in redis. The data collection process is shown in Figure 4. Data index mapping overview is shown in Figure 5.

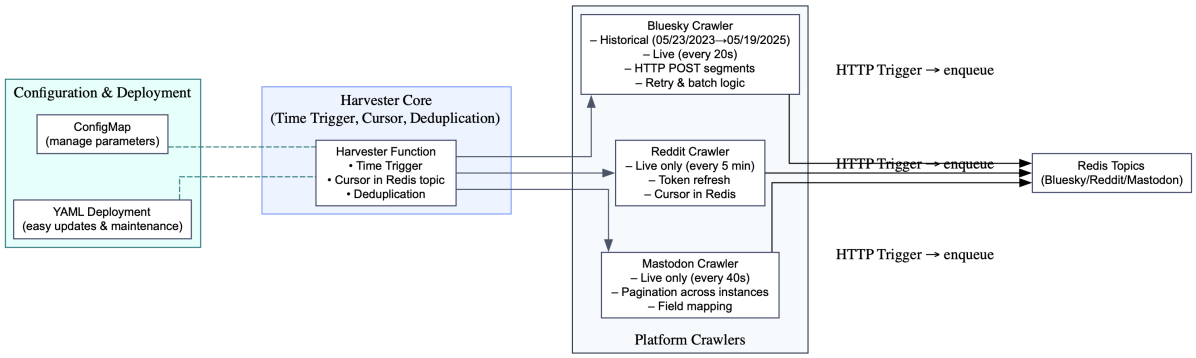

Figure 4: Data collection

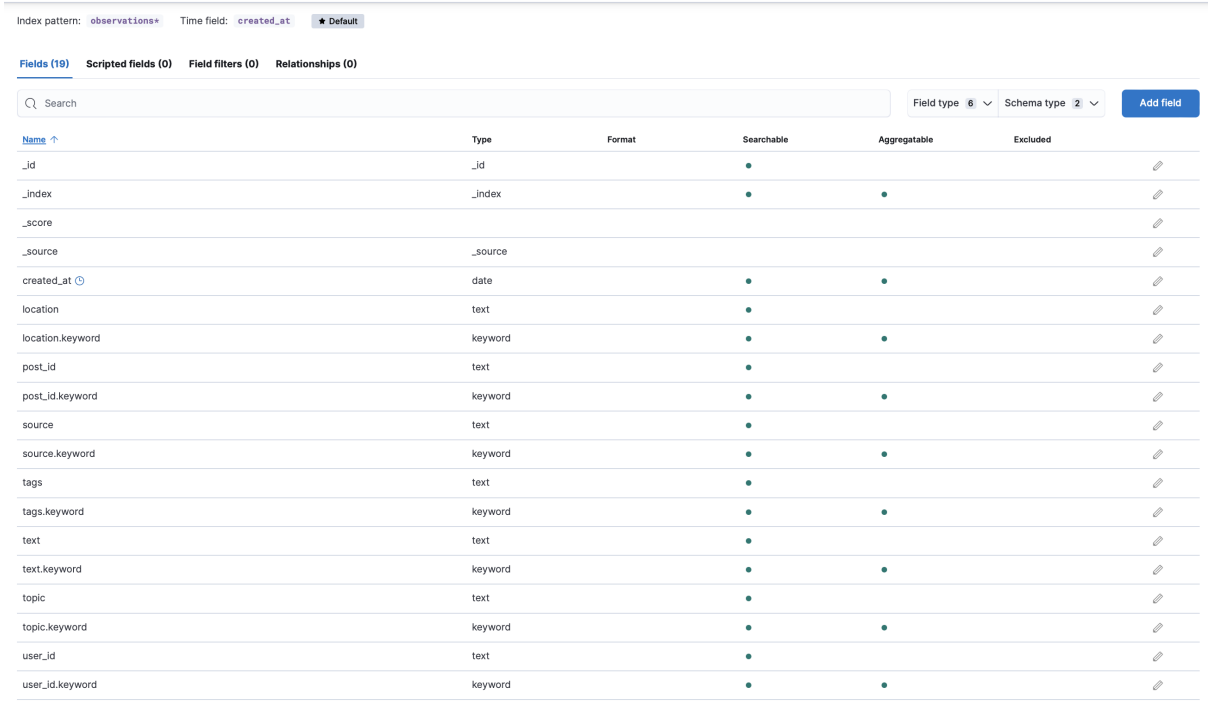

Figure 5: Index Mapping Overview

2.3.1 Harvester

Our Data Collection starts with three harvester with each one responsible for one specific social Media

1. **Bluesky**: On the Bluesky platform, historical data is collected through a time-triggered crawler that retrieves all posts from May 23, 2023, to May 19, 2025.

Starting from May 19, 2025, a real-time crawler runs at 10-second intervals to capture new posts. Both crawlers utilize the Atproto protocol to establish a connection with the Bluesky client. Data harvesting is performed iteratively using three key political entities - Labor, Coalition, Greens, and their respective leaders as query parameters to gather relevant posts. All retrieved content is stored in full and sent in batches to an enqueue function via HTTP POST to manage memory usage effectively. To support pagination, Bluesky provides a cursor for each query, which is used to track the current position in the result set. These cursors are stored in Redis under corresponding topics, enabling the system to resume from the last collected point and ensure comprehensive historical coverage.

2. **Reddit**: Due to Reddit's limitations on data crawling frequency, only a real-time harvester with praw is used to capture the most recent posts. This crawler was initiated on May 19, 2025, and is triggered every five minutes to retrieve data from selected subreddits, including those related to major Australian cities and election topics. Retrieved posts are filtered based on whether election-related keywords appear in their titles or content. To maintain continuity in data collection, a cursor for each query is stored using Redis, which records the last processed item and ensures previously crawled content is skipped. Since the return value from reddit is not in JSON format, to make it compatiable with Redis requirement, only necessary information are extracted from the post and stored in JSON format and sent to enqueue function use HTTP POST as well.
3. **Mastodon**: For Mastodon, due to platform limitations, data harvesting is scheduled to run every 40 seconds starting from May 19, 2025. During each cycle, the harvester queries the paginated API endpoints of selected instances to retrieve the latest content.To prevent duplicate data caused by overlapping queries or repeated posts across different instances, a list of previously harvested post IDs is maintained in Redis. This list is used to filter out any duplicates before processing. The extracted content is formatted as JSON and forwarded to the enqueue function via an HTTP trigger for further processing.

2.3.2 Enqueue

The enqueue function serves as an intermediary that places data onto a Redis queue, effectively decoupling the harvesters from the processors in our system. This design enables an event-driven architecture where harvesters and processors can be developed, deployed, and scaled independently.

In our implementation, each harvester sends an HTTP POST request to a predefined endpoint, which is handled by the HTTP trigger (route) associated with the enqueue function. The function extracts the payload from the request body and stores it in the appropriate Redis list (queue) based on the topic specified in the X-Fission-Params-Topic HTTP header. This approach ensures flexible, modular data ingestion and routing for downstream processing.

2.3.3 Processor

Once data is stored in any of the three Redis topics, the corresponding Message Queue Triggers are activated immediately. These triggers invoke the appropriate processor functions, which extract and transform the raw posts into structured JSON format. For location extraction, we adopt a rule-based approach: the processor scans the post content for keywords associated with Australian locations. If a match is found, the corresponding state name is assigned as the location field. Additional metadata such as post time, post ID, user ID, and platform information are also included. After processing, the enriched and structured post is pushed to the observations topic in Redis. This final output is then ready to be ingested into ElasticSearch for indexing and analysis.

2.3.4 Addobservations

The data collection pipeline concludes with a unified function called addobservations. This function is triggered by a Message Queue Trigger that monitors the observations topic in Redis for new entries. When triggered, the function retrieves the processed data from the queue and inserts it directly into ElasticSearch. The connection to ElasticSearch is established using the appropriate credentials, ensuring secure and reliable data ingestion. This final step completes the pipeline, making the data available for search, analysis, and visualization.

This project has achieved the automated capture, real-time processing and visual support of data from the three major social platforms through the serverless architecture of Kubernetes + Fission. ConfigMap is adopted to manage variable parameters, YAML deployment is used to simplify operation and maintenance, and optimizations have been made in links such as crawling, deduplication, rate limiting and batch processing to ensure the stability and scalability of the system. The processed data flows through the Redis message queue and Elasticsearch storage. Eventually, it provides a foundation for front-end analysis and display through the RestFUL API, and builds a complete, efficient and maintainable pipeline for social media content analysis. This modular and rate-aware collector strategy ensures reliable and balanced data collection, achieving near-real-time processing without occupying platform apis or our own system resources.

2.4 Scalability Considerations

To meet the demands of large-scale, real-time election monitoring, our system is architected with scalability as a core principle. Each layer—from data harvesting to enrichment and indexing—is designed to scale horizontally, ensuring consistent performance under variable workloads.

1. **Parallel Harvesters**: Each harvester operates independently and can be duplicated across multiple Mastodon instances, subreddits, or keyword sets. This allows for concurrent scraping across varied data sources without creating bottlenecks or API overuse.
2. **Stateless Fission Functions**: The enrichment layer, built on Fission, is fully stateless. Each function is triggered via MQTrigger on new Redis queue entries and

can run in parallel across multiple pods. This ensures rapid, distributed processing of incoming data, with Kubernetes automatically balancing load (Fission Authors, 2024).

3. **Redis as a Scalable Buffer**: Redis acts as a queue-based buffer that isolates the harvesting and processing layers. Under high ingestion rates, Redis absorbs traffic spikes and smooths variability without overloading the enrichment or indexing pipelines (Redis Labs, 2024).
4. **Autoscaling via Kubernetes**: Both harvester and processor pods are deployed using Kubernetes Helm charts with autoscaling enabled . CPU and memory usage thresholds are continuously monitored to scale pods up or down, ensuring elastic provisioning without resource waste(Kubernetes Authors, 2024).

Together, these mechanisms allow the system to adapt gracefully to election season surges, new platform integrations, and evolving analytical workloads, maintaining responsiveness while preventing overload or downtime.

2.5 Fault Tolerance

Given the asynchronous and distributed nature of our architecture, fault tolerance is not an afterthought but an embedded feature at every layer of the system. Each component is designed to recover gracefully from partial failures without compromising the continuity or consistency of the data flow.

1. **Harvesting Resilience**: All harvesters are designed with retry logic to handle transient network failures and platform-side rate limitations. For instance, Mastodon and BlueSky harvesters implement backoff strategies when encountering 429 or timeout responses. Historical crawlers also maintain local cursor checkpoints to resume from the last successful position.
2. **Redis Queue Isolation**: Redis serves as an intermediary layer that decouples data producers and consumers. This ensures that a temporary failure in either harvesting or processing does not halt the other. Each post is stored with a time-to-live (TTL), preventing stale data accumulation (Redis Labs, 2024).
3. **Function-level Isolation via Fission**: Each processor function is invoked in a stateless environment. Failures in one function (e.g., location extraction or sentiment scoring) do not propagate to others. Logs are emitted per invocation, making it easy to isolate and diagnose errors without restarting the pipeline (Fission Authors, 2024).
4. **ElasticSearch Buffering and Retry**: If the indexing pipeline encounters connection errors or receives malformed records, a fallback retry mechanism is triggered. Faulty documents are logged into a quarantine queue for later inspection and reinjection (ElasticSearch Authors, 2024).
5. **Container Recovery and Monitoring**: Kubernetes continuously monitors the health of each pod. Crash-looping pods are automatically restarted. Moreover, alerts and metrics are routed through Prometheus-compatible exporters, allowing us to track failure patterns in real time.

These design choices collectively ensure that no single point of failure can compromise the integrity of the system. Even under conditions such as API outages, memory surges, or malformed payloads, the system continues to operate in a degraded-but-recoverable mode, preserving pipeline consistency and uptime.

3 System Implementation

3.1 Technology Stack

The implementation of our election sentiment analytics platform is grounded in a microservices-oriented stack, built on open-source technologies for scalability, resilience, and reproducibility. The system is implemented in Python, orchestrated via Kubernetes, and fully containerised for modular deployment. Technology stack architecture flowchart is shown in Figure 6.

The key technologies used are as follows:

1. **Python**: All harvesters, enrichment functions, and RestFUL API services are written in Python 3.9. Its versatility and rich ecosystem (e.g., 'requests', 'praw', 'pydantic') support everything from API integration to data transformation and inference.
2. **Redis**: A lightweight, in-memory message queue that acts as a buffer between the harvesting and processing stages. Redis supports high-throughput queuing and topic-specific keying, enabling event-driven processing without bottlenecks.
3. **Fission (Serverless Framework)**: Stateless, reactive processing of posts is handled by Fission functions triggered by Redis queues. Fission enables fast deployment and isolated execution of enrichment modules (e.g. location inference).
4. **ElasticSearch**: All enriched posts are indexed in a structured JSON format within an ElasticSearch cluster. Mappings are defined for fields such as 'topic', 'location', 'created_at', and 'source', allowing filtered and geo-aware querying.
5. **RestFUL API**: The RESTful interface is implemented using Flask, exposing endpoints that allow querying election-related analytics from ElasticSearch. Core endpoints include retrieving number of posts under a given date or topic. Inputs are received through URL path parameters, and the API returns structured JSON responses. This abstraction hides the complexity of the underlying ElasticSearch queries, enabling front-end tools and notebooks to consume and visualize the data with minimal overhead (Grinberg, 2018).
6. **Kibana**: Used for interactive data visualisation, Kibana connects directly to the ElasticSearch index and presents time-series plots, tag clouds in a web dashboard (Kibana Authors, 2024).
7. **Kubernetes (OpenStack NeCTAR)**: All services are containerised using Docker and deployed to a NeCTAR-hosted Kubernetes cluster. Helm charts are used to configure, deploy, and manage autoscaling policies, volume mounts, service endpoints, and RBAC access.
8. **Helm + YAML**: Helm provides templating for Kubernetes resources, while custom YAML definitions are used for deploying Redis, ElasticSearch, Kibana, Fission,

and API Gateway services. Secrets and credentials are managed via environment variables and Kubernetes secrets.

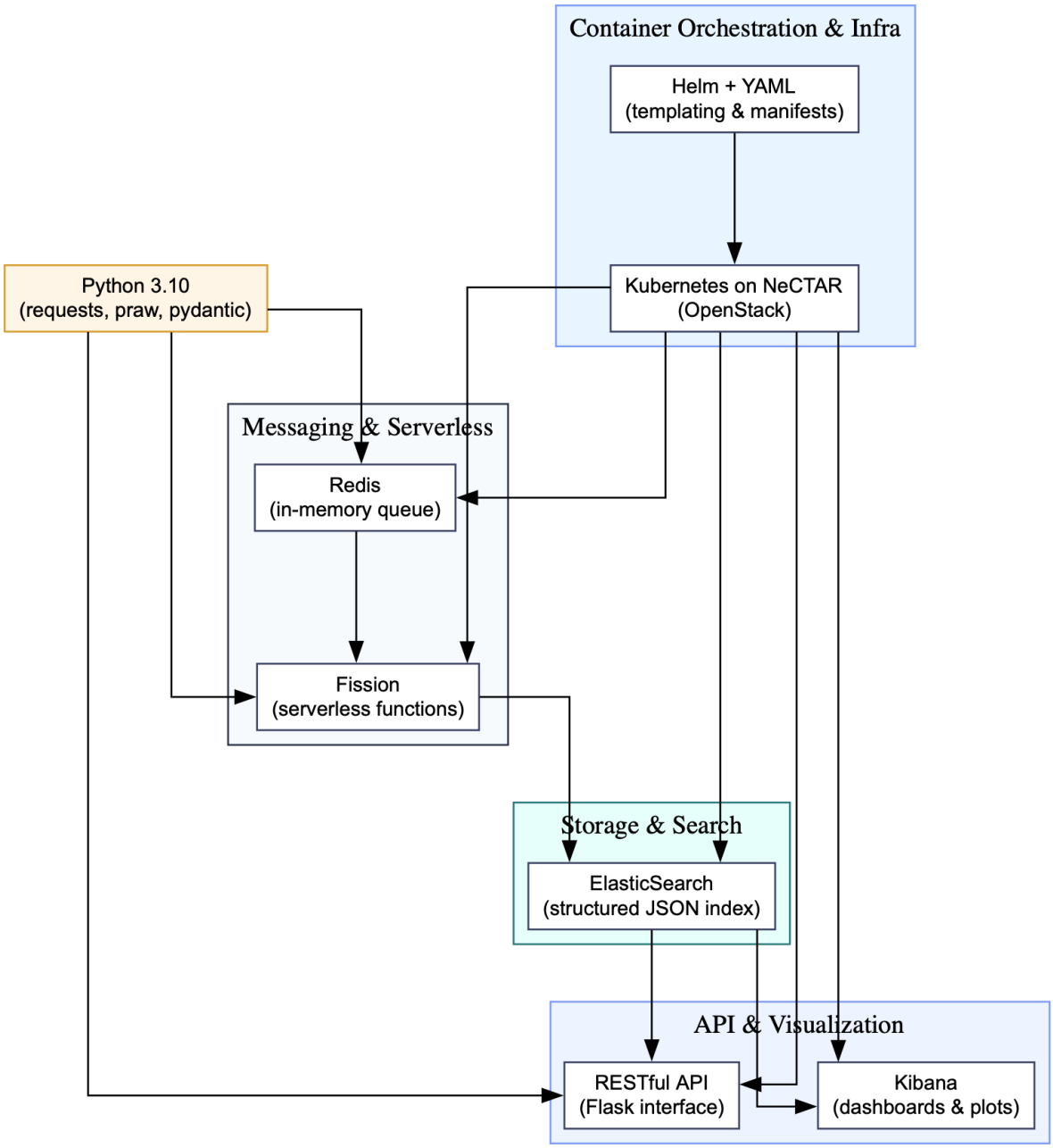

Figure 6: Technology Stack architecture flowchart

This stack was selected for its compatibility with public research cloud constraints, community support, and ability to support reproducible infrastructure-as-code practices. All components are deployed within dedicated Kubernetes namespaces to enable clear isolation between system stages.

3.2 Component Configuration

Each component of the system was configured to operate as an independent and reusable service within the Kubernetes cluster. The deployment strategy emphasizes clear separation of concerns, environment-specific modularity, and the ability to scale or update individual components without disrupting the broader pipeline.

1. **Harvester Configuration**: Each harvester (Mastodon, Reddit, BlueSky) is deployed as a scheduled job pod, running a time-triggered Python script. CronJob configurations within Kubernetes schedule executions at intervals suited to each platform's API constraints (e.g., 20–60 seconds for Mastodon, 5 minutes for Reddit). Keywords, platform-specific cursors, and queue targets are defined through config maps and passed into each pod as environment variables.
2. **Redis Deployment**: Redis is deployed via a Helm chart in a dedicated namespace. It operates in a single-node configuration with persistence disabled for lightweight buffering. Key names are dynamically structured as `<topic>_<platform>_posts` to ensure isolation by both data source and analytical context. TTL is set to auto-clean old entries that are not consumed within a fixed window.
3. **Fission Setup**: Fission was installed into the cluster using its official Helm chart. A Fission environment was registered for Python 3.10, and functions were deployed for tasks including: sentiment scoring, topic classification, location inference, and payload validation. MQTrigger bindings were configured to link specific Redis queues to corresponding function endpoints, allowing automatic invocation upon new message arrival.
4. **ElasticSearch Configuration**: ElasticSearch is deployed in a three-node replica cluster using the Bitnami Helm chart. The index `social_posts` is created with custom mappings for key fields such as:
   - `created_at` — date type
   - `sentiment` — float type
   - `tags, topic, source, location` — keyword types
   - `text` — text type with English analyzer

   A separate alias is maintained to support version control and safe reindexing during model or field updates.
5. **Kibana Interface**: Kibana is exposed via a NodePort service and configured to point to the active ElasticSearch index. Dashboards are built with standard visualisation widgets including time series graphs, pie charts for topic distribution, and world maps for geospatial sentiment.
6. **RestFUL API Deployment**: This Flask RestFUL API is containerized and deployed in the Kubernetes cluster, and exposed externally through Fission functions and HttpTriggers. Among them, the core endpoints all initiate HTTP calls through Python's `request` library and combine the `logging` module for basic request and error logging and handling. The connection parameters of Elasticsearch are injected through environment variables, and timeout and certificate verification options are uniformly set on HTTP calls to ensure the stability and security of the interface.

7. **Helm + Namespace Separation**: All services are installed using Helm in logically isolated namespaces: `harvester-ns`, `analytics-ns`, `storage-ns`, and `ui-ns`. This enables clean teardown and targeted updates without affecting unrelated components. Each deployment is version-controlled and stored in a Git repository with YAML templates for reproducibility.

These configurations collectively support modular updates, reproducibility, and efficient resource allocation, while ensuring each component communicates reliably and scales as required within the distributed pipeline.

3.3 RestFUL API and Access Layer

To enable structured access to harvested and enriched election data, we designed and deployed a RestFUL interface using Flask. This layer allows external tools — such as data dashboards, analysis notebooks, or automated monitoring scripts — to query, filter, and summarise the indexed posts from ElasticSearch.

1. **Framework and Deployment**: The RestFUL API is implemented using Python and Flask. The reason why we chose Flask is mainly because its lightweight feature greatly simplifies the entry threshold of the project: Flask only provides the most basic routing and request processing functions, and does not force the introduction of excessive dependencies or complex project structures. This enables us to quickly set up a usable HTTP service with very little code.
2. **Endpoint Design**: The API is built around three resource endpoints:
   - `GET /es-api/indices`: Extract the `index` field in each returned record and return all index names in list form.
   - `GET /es-api/scroll-search`: The Scroll based on Elasticsearch realizes the paging query of documents with a large amount of data. Start the scrolling context at the first request and pick up a maximum of 1000 entries. In addition to the data array, the function will also return the current `scroll-id` for the client to continue pagination and pulling. This service will construct the corresponding query load, return the `source` array that matches the documents, and handle requests and errors uniformly at the same time.
   - `GET /es-api/health`: Obtain the cluster health metrics and return the original JSON data.

   All endpoints accept query parameters via URL query parameters and retuf lflrn JSON-structured responses suitable for dashboard consumption.
3. **Build and Deployment**: Using the build.sh script, during the build phase, all dependencies and source code will be copied into the deployment package together to ensure that the function image can be run directly. And pre-configure the resource quota, concurrency degree and environment variables in the YAML of the Function, so that the deployment can be automated and the configuration can be reused.
4. **Integration with ElasticSearch**: At the back end, each route initiates a synchronous HTTP request through the requests library and sets a 30-second timeout. Extract the index name, document source array or health metric in the response

and uniformly encapsulate them as JSON for return. This layer of abstraction shields the native query syntax and can maintain the stability of the interface when the underlying index structure changes.

The RESTful layer forms a key part of our system's usability and interoperability, enabling data access for researchers, decision-makers, and future automated analysis pipelines — all without requiring direct interaction with ElasticSearch query syntax.

3.4 Testing Outcomes and Significance

The testing framework demonstrates comprehensive coverage and maintainability, ensuring the system's operational integrity and scalability.

Key functionalities tested include data transformation accuracy, message queue efficiency, and storage reliability, with particular attention to error handling for malformed inputs and service failures.

The implementation leverages mocked Redis and Elasticsearch clients to simulate real-world interactions, while configuration management ensures adaptability across environments. Performance evaluations confirm the pipeline's throughput and storage operation reliability, critical for large-scale data processing.

The test suite's design facilitates future extensions, allowing seamless integration of new test cases as the system evolves. By validating reliable data harvesting, accurate processing, efficient queuing, and secure storage, this testing framework establishes a robust foundation for the Cluster Cloud Computing Project, ensuring its functionality aligns with project objectives and academic standards. an end-to-end (E2E) test covering the core pipeline.

3.5 Front End and Visualisation

The Front End and Visualisation of this system is constructed using jupyter notebook, integrating multiple Python data analysis and visualization libraries to achieve multi-dimensional and interactive visual charts, and supporting the exploration of the sentiment analysis of various political parties towards the general election on social media. The main functions include:

(1) Use Matplotlib and Seaborn to generate charts related to the number of posts, including annual bar charts and monthly line charts, to analyze the Posting frequency and trends of each political party.

(2) In the sentiment analysis section, Plotly is used to create a quarterly sentiment mean change chart (interactive line chart), and a box plot is drawn through Seaborn to visually display the sentiment distribution range and bias of each political party.

(3) Word cloud analysis: After removing stop words with wordcloud and NLTK, high-frequency word clouds are generated. Time and party filtering are achieved through ipywidgets, and interactive updates are supported.

(4) The geographic visualization function is implemented based on Plotly's choropleth graph. By integrating the GeoJSON boundary data of each state in Australia, it displays

the daily sentiment scores of each state. At the same time, it provides dual filtering interaction functions of party affiliation and time.

(5) Plot the kernel Density Estimation (KDE) graph through Seaborn.kdeplot to show the emotional distribution of each political party on the three platforms.

(6) Aggregate the emotion scores into a partisan - platform two-dimensional matrix through pandas groupby, visualize the mean differences with Seaborn.heatmap, and draw heat maps to highlight the preference characteristics between platforms.

(7) By calculating the Posting frequency of users, they are classified into latent users, medium-frequency users and highly active users, and the emotional characteristics of different activity groups are compared by box plot to reveal the potential relationship between user participation and emotional expression.

The entire front-end analysis module maintains a loosely coupled design with the back-end in structure to ensure that the charts can be independently debugged and expanded.

4 Supported Scenarios and Analytical Results

4.1 Scenario 1: How does the sentiment of different regions for different parties change since 2022 election?

4.1.1 Descriptions

This scenario aims to analyze the changing trends of public sentiment towards different political parties in various geographical regions since the 2022 Australian federal election. The research objective is to understand whether the supportive or hostile attitudes of each state towards major political parties (such as the Liberal Party, the Labour Party, the Green Party, etc.) have changed significantly over time. This study takes the relevant posts on three platforms as the data source, covering the comments and posts made by Australian users at different time periods and under different political party tags, and processes them through natural language processing and sentiment analysis models.

To achieve the above analytical goals, we collected post data containing key words of major political parties based on three platforms, combined with the fields of post time, political party tags and geographical location. To comprehensively reflect the activity and emotional fluctuations of political party-related discussions in different time periods and among different states, we have designed and implemented multiple interactive visual charts. The dynamic front-end interface was constructed using Python visualization and data processing libraries such as Plotly, pandas, GeoPandas, and matplotlib.

Firstly, to measure public participation, we designed a chart showing the changes in the number of posts, presenting the trend of the total number of posts made by each political party each month through bar charts and line graphs, which helps us understand the activity level and fluctuation cycle of each political party in public discussions. Secondly, in order to track the dynamic change trend of public sentiment, we averaged and smoothed the scores of political party sentiment on a quarterly basis, and plotted the changes in sentiment trends through line graphs to show the direction of political party sentiment over time.

To further depict the range of emotional fluctuations, we constructed a box plot to visualize the emotional distribution of different political parties at different time periods. This plot reveals statistical characteristics such as emotional extremes, upper and lower quartiles, and median. To assist in the semantic understanding of emotional content, we have implemented an interactive word cloud map using ipywidgets and the WordCloud library. Users can filter data by time range and political party to display keywords, thereby intuitively observing the changes in discussion tags and focus of attention of different political parties at different times.

Finally, to explore the differences in emotions in the geographical dimension, we constructed a geographical emotion distribution map based on the state boundary data of Australia, and displayed the emotion scores of each state towards different political parties on a certain day through color coding. This graph is implemented by combining GeoJSON geographic boundary data with Plotly Choropleth Mapbox, supporting the interactive screening operation of political parties and time, and helping to study the differences in political sentiment among regions and their evolution process.

4.1.2 Graphical Results and Analysis

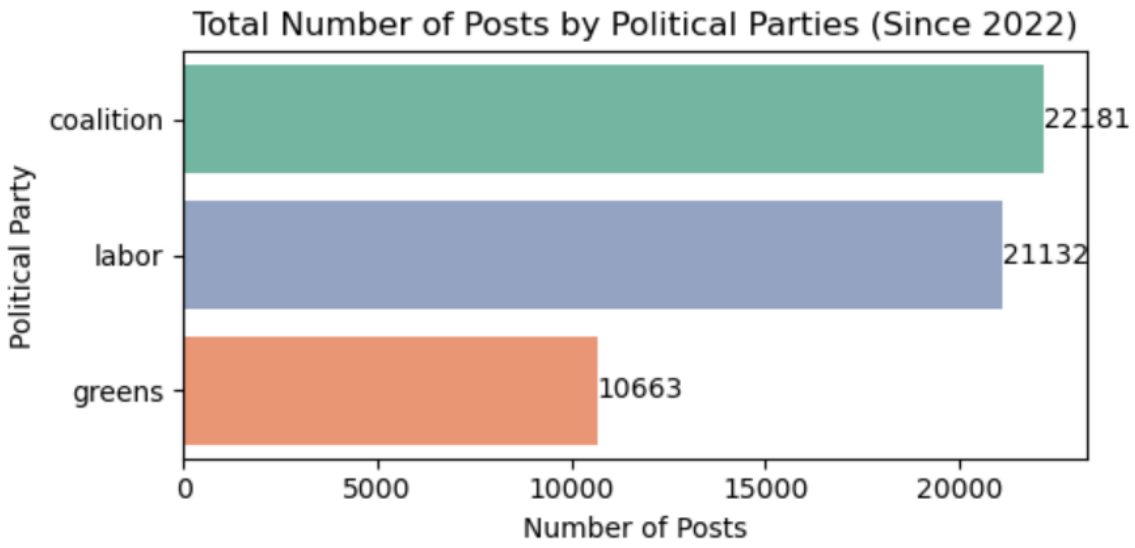

Figure 7: **Total Number of Posts by Political Parties (Since 2022)**

This bar chart (Figure 7) shows the cumulative number of posts by the three major parties on social platforms since the 2022 Australian federal election to measure the public's discussion of each party. Among them, Coalition had the highest number of posts, reaching 22,270. Following closely behind is Labor, with a total of 21,199 articles; However, the number of posts on Greens is significantly lower, at only 10,463. In terms of magnitude, the gap between Coalition and Labor is not significant, both remaining above 20,000, reflecting that these two major parties have continuously attracted a great deal of public attention and discussion since the election. In contrast, the discussion voices on Greens are relatively weak, with the number of posts being less than half of

the previous two, indicating that in the overall public opinion environment, the attention and participation in Greens are relatively low. This preliminary result provides a basis for the subsequent in-depth analysis of the emotional change trends in different states and different time periods.

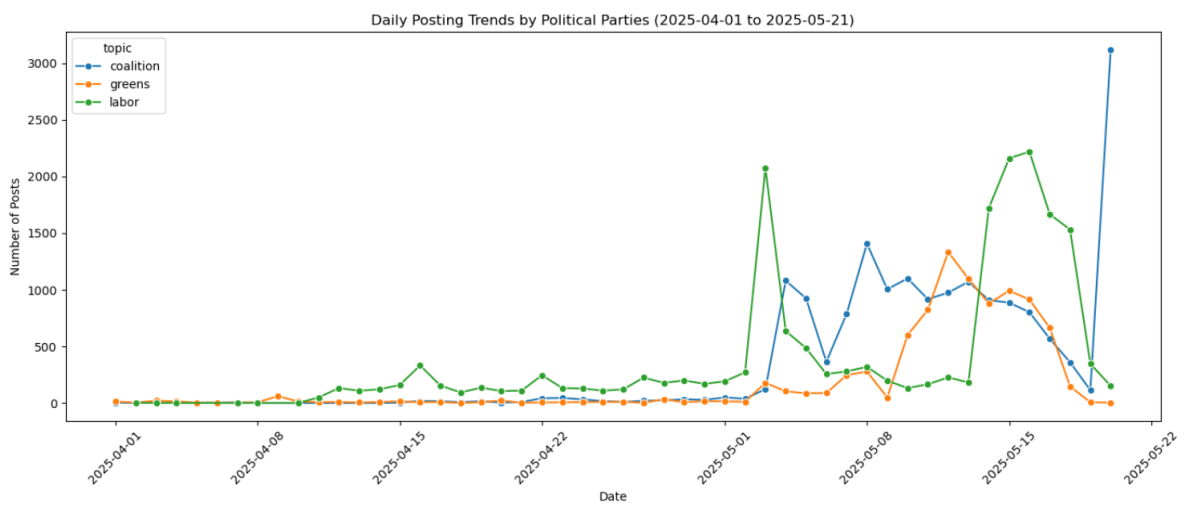

Figure 8: **Daily Posting Trends by Political Parties (2025-04-01 to 2025-05-21)**

This line chart (Figure 8) shows the daily Posting volume changes of Coalition, Labor and Greens on social media platforms from April 1 to May 21, 2025. After entering May, the discussion heat rose significantly: On May 5th, Labor suddenly climbed to approximately 2,080 posts, and then on May 6th, Coalition also saw a breakthrough growth, reaching about 1,100 posts. Subsequently, the discussion volume of both sides gradually dropped back to the normal level day by day. Subsequently, Labor reached its second peak again on May 15th, with a peak of approximately 2,220 cases, significantly higher than the levels of both parties during the same period. Meanwhile, Greens and Coalition gradually dropped back to the medium range. On May 21st, Coalition once again experienced an abnormal surge, with the number of posts reaching approximately 3,100, setting a new record for the entire period. Meanwhile, the number of posts by Labor and Greens both dropped to low levels on that day. Such huge single-day fluctuations often correspond to large-scale public debates, breaking news or official statements and other events, highlighting the strong driving effect of major time points on the public opinion volume of various political parties and camps.

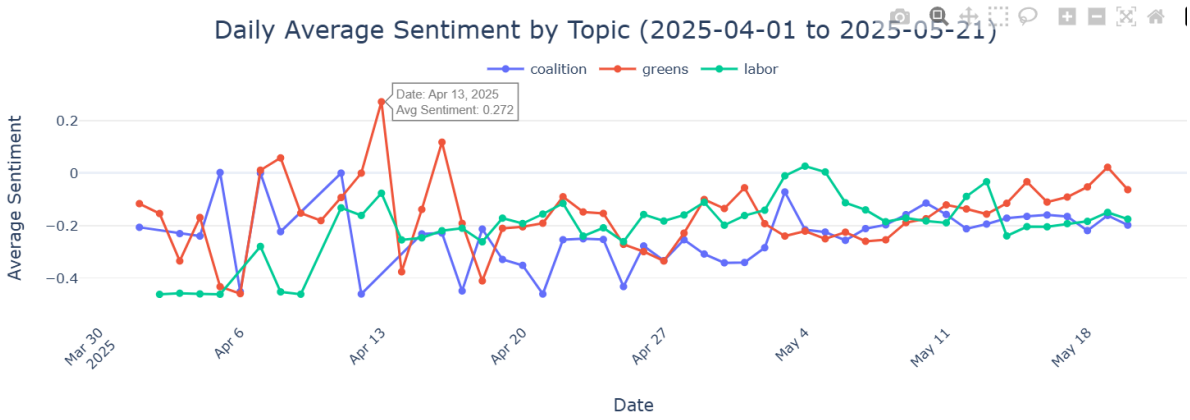

Figure 9: **Daily Average Sentiment by Topic (2025-04-01 to 2025-05-21)**

This line chart (Figure 9) shows the daily average emotional score curves of the three major political parties on social media platforms from April 1 to May 21, 2025. By hovering the mouse over the curve, one can view the specific date and score value corresponding to each node, thereby intuitively perceiving the short-term fluctuations of the sentiments of each political party. Overall, most of the three curves are distributed in the neutral to negative range, reflecting the relatively cautious and even slightly critical discussion atmosphere of the public during this period. As for Coalition, its emotional trend is mainly negative, and the occasional positive discussions are often difficult to sustain. The emotional curve of Greens is more volatile, suggesting that environmental protection topics have gained more resonance on specific days. By contrast, Labor's emotional score was relatively stable.

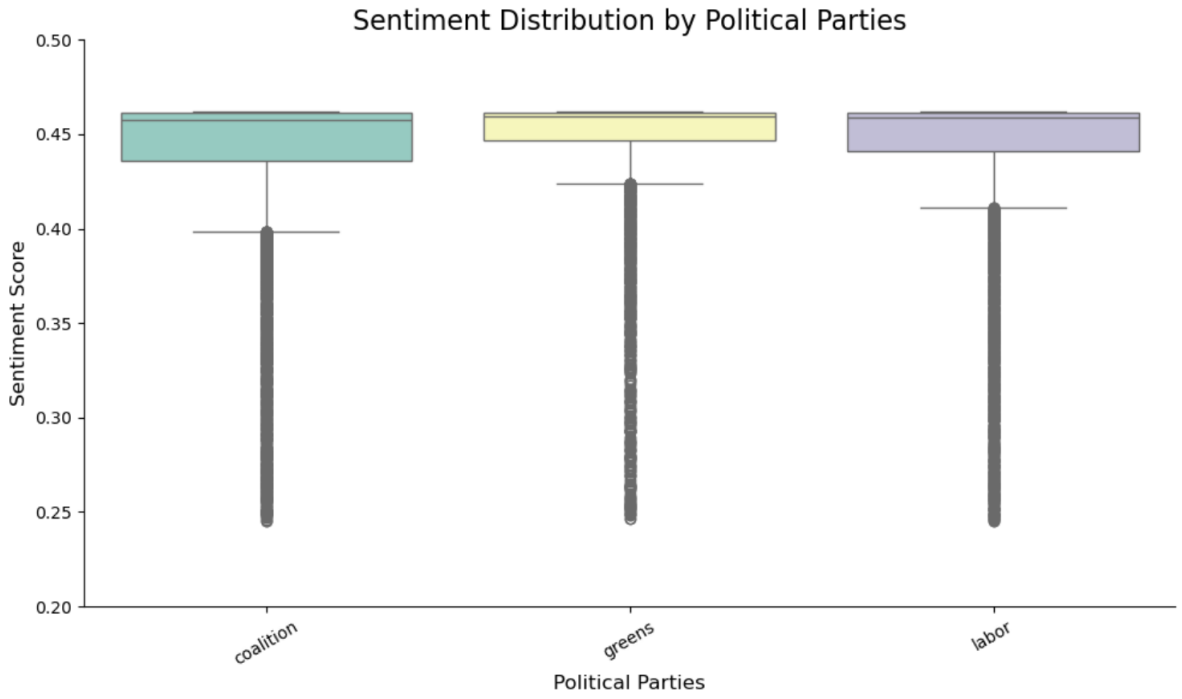

Figure 10: **Sentiment Distribution by Political Parties**

This box plot (Figure 10) shows the distribution characteristics of all the sentiment scores of the crawled posts for Coalition, Greens and Labor since the 2022 general election. The median values of the three parties were all concentrated between approximately 0.45 and 0.47, indicating that most of the discussions presented moderately positive emotions. However, their IQR codes were all very narrow, suggesting that the emotional scores were mainly concentrated within this narrow range. whisker extends downward to approximately 0.40, while the outlier outside the box is as low as about 0.25, reflecting that public sentiment may experience significant negative peaks in a few time periods. However, the upward extension is relatively limited, with fewer extreme positive emotions, and the score is basically no more than 0.48. The shapes and positions of the three boxes are highly similar, meaning that although the discussion volumes and time periods of different parties vary, the overall emotional intensities are not much different. The main fluctuations came from a small number of extremely negative comments rather than large-scale enthusiastic support.

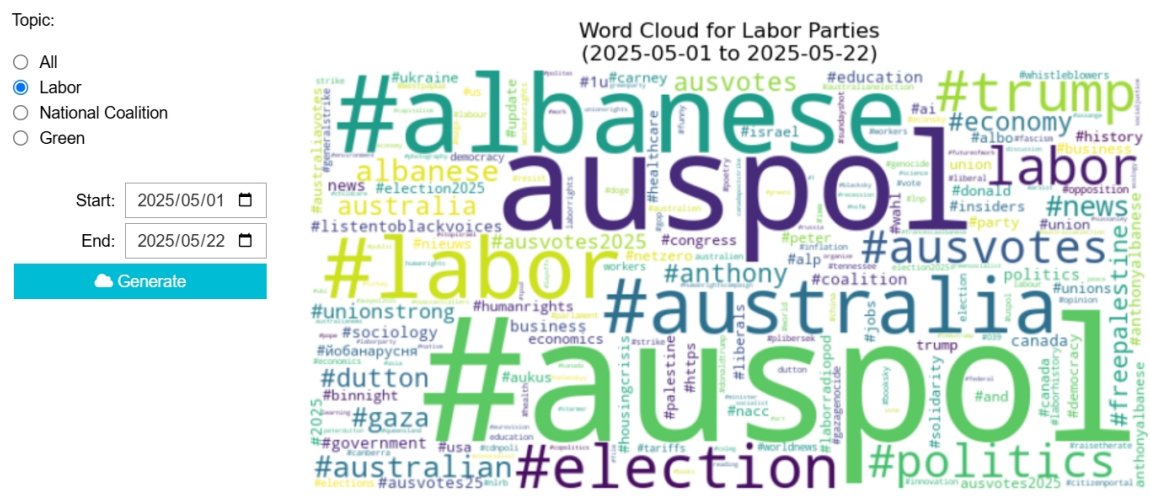

Figure 11: **Word cloud for labor parties**

Figure 12: **Word cloud for all parties**

The two word (Figure 11 and 12) cloud images above are interactive word clouds generated for discussions between all political parties and Labor from May 1st to May 22nd, 2025, showing the commonalities and differences in topic concerns among different political parties. From the perspective of "all political parties", election keywords such as #auspol, #albanese, #labor, #coalition, and #election dominate, and international issues such as #trump, #gaza, and #ukraine are also widely mentioned. It indicates that public opinion attaches equal importance to local politics and global events. From the perspective of Labor, in addition to continuing the above-mentioned election situation tags, keywords such as #unionstrong, #economy, and #freepalestine are more prominent, reflecting that its supporters are more concerned about issues such as economic livelihood, trade union rights, and social justice, presenting a more distinct integration of issues and value orientation.

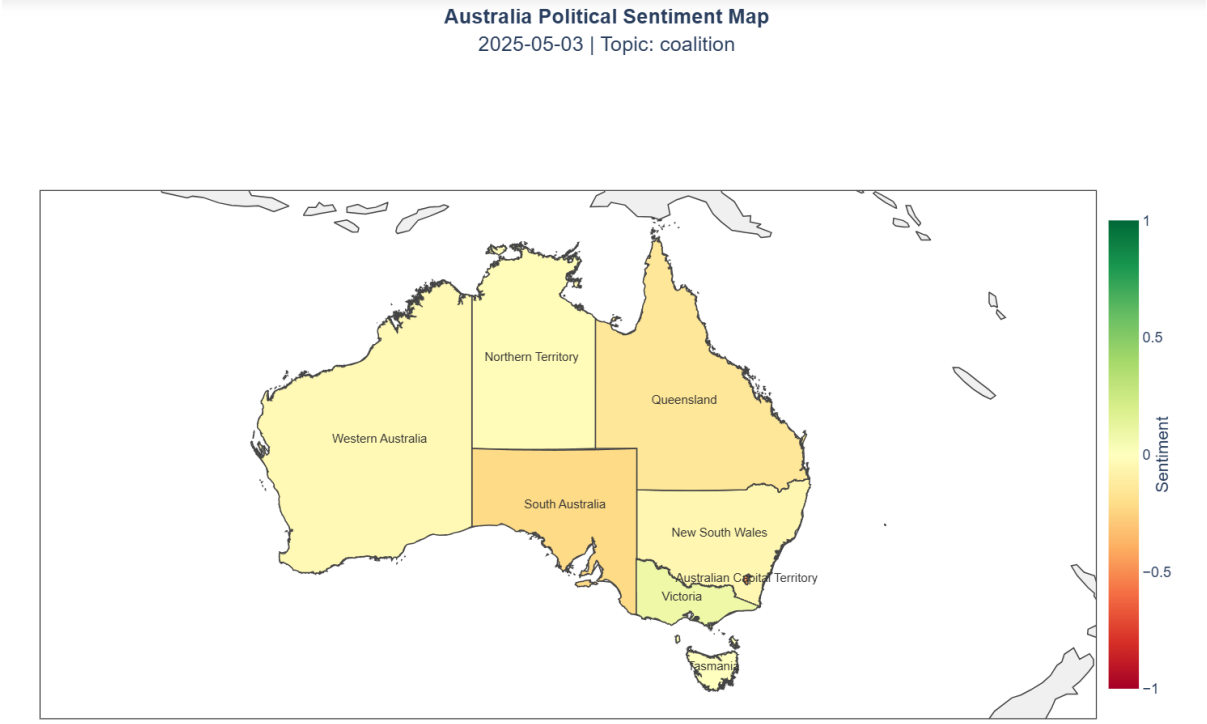

Figure 13: **Australia political sentiment map(coalition)**

The two maps (Figure 13 and 14) below respectively show the distribution of public sentiment of Labor and Coalition in each state on May 3, 2025. The map encodes the emotion score through color gradations ranging from dark red to dark green. As for Labor, New South Wales showed the strongest positive sentiment, while Queensland experienced a slight negative one, reflecting that the discussions about Labor among users in the state on that day were somewhat reserved. In contrast, on the same day, the sentiment towards the Coalition was negative in many states. The mood was the lowest in Queensland and South Australia, approaching -0.5 (orange) and -0.7 (light red) respectively, indicating strong criticism or dissatisfaction. While Victoria is slightly positive (light green). This comparison indicates that at the same time, there are significant differences in the distribution of public sentiment among different political parties in different regions. Labor performed particularly strongly in New South Wales and Victoria, while Coalition's support enthusiasm was significantly insufficient in Queensland and South Australia, highlighting the spatial heterogeneity of regional election situations and public opinion sentiment.

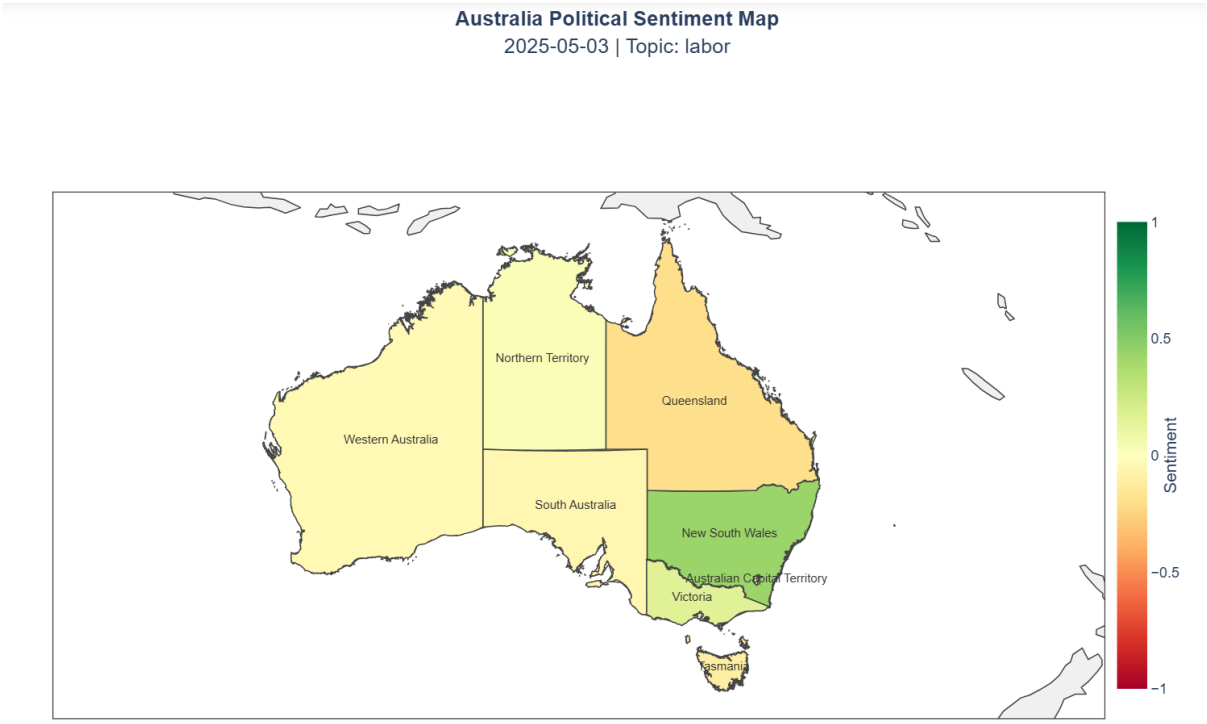

Figure 14: **Australia political sentiment map(labor)**

Summary: Regarding Scenario 1, we have revealed through multi-dimensional visualization the trends of public opinion heat and emotional evolution of different political parties in various regions of Australia since the 2022 federal election. Overall, the public's attention to Labor and Coalition is significantly higher than that to Greens. The number and popularity of posts by the three parties on social media platforms show cyclical fluctuations, often related to policy releases or news events. Sentiment analysis shows that the overall discussion sentiment is mainly neutral to negative, and the outliers of negative comments are frequent, indicating that the public generally holds a cautious or even critical attitude towards political issues. There are significant differences in the distribution of emotions in different regions. For example, Labor often receives higher positive feedback in New South Wales and Victoria, while Coalition has more prominent negative evaluations in Queensland and South Australia. The word cloud further shows that different political parties focus on different topics. The mainstream parties revolve around the election situation, while Labor not only focuses on the election situation but also deeply integrates economic, trade union, social justice and international human rights issues. Overall, Australian voters present a complex emotional structure and focus of concern in regional and temporal dimensions, providing important clues for understanding the evolution of voters' attitudes.

4.2 Scenario 2: Is the sentiment of Australians for the particular parties the same across Mastodon, BlueSky and Reddit?

4.2.1 Descriptions

This scene aims to explore whether there is consistency in the sentiment of the Australian public towards major political parties on different social platforms (Mastodon, BlueSky and Reddit). The research objective is to analyze the differences in the emotional distribution of the same political party on different platforms, and thereby reveal the potential impact of the platform ecosystem on the expression of political emotions. This study is based on user Posting data since the 2022 federal election, uses a sentiment analysis model to calculate the sentiment score of the text, and conducts a comparative analysis with party tags and platform sources as the core variables.

To achieve the above analytical goals, we first use sentiment analysis tools to score the text and construct kernel density estimation maps (KDE maps) to show the sentiment distribution profiles of each political party on different platforms one by one. This graph can be filled with colors to show the similarities and differences in the distribution among platforms, helping to identify on which platforms a certain political party is more likely to obtain positive or negative emotional expressions.

Secondly, we constructed an emotional score box plot to summarize and distribute the emotional data of each political party on three platforms. This chart reveals the concentration degree and extreme value changes of emotions on each platform, facilitating the comparison of the overall emotional tendencies of political parties on different platforms.

To further strengthen the emotional correspondence between platforms and political parties, we use heat maps to show the average emotional scores of each political party on different platforms. Reflecting positive or negative tendencies through the depth of colors can help quickly identify the platform party combinations with significant emotional differences.

In addition, we have introduced the dimension of user activity, classifying users into three categories based on their Posting frequency: "low frequency", "medium frequency", and "high frequency", further constructing an emotional box plot that intersects user stratification and the platform. This graph is used to analyze whether there are systematic differences in emotional expression styles among users with different levels of participation on different platforms.

4.2.2 Graphical Results and Analysis

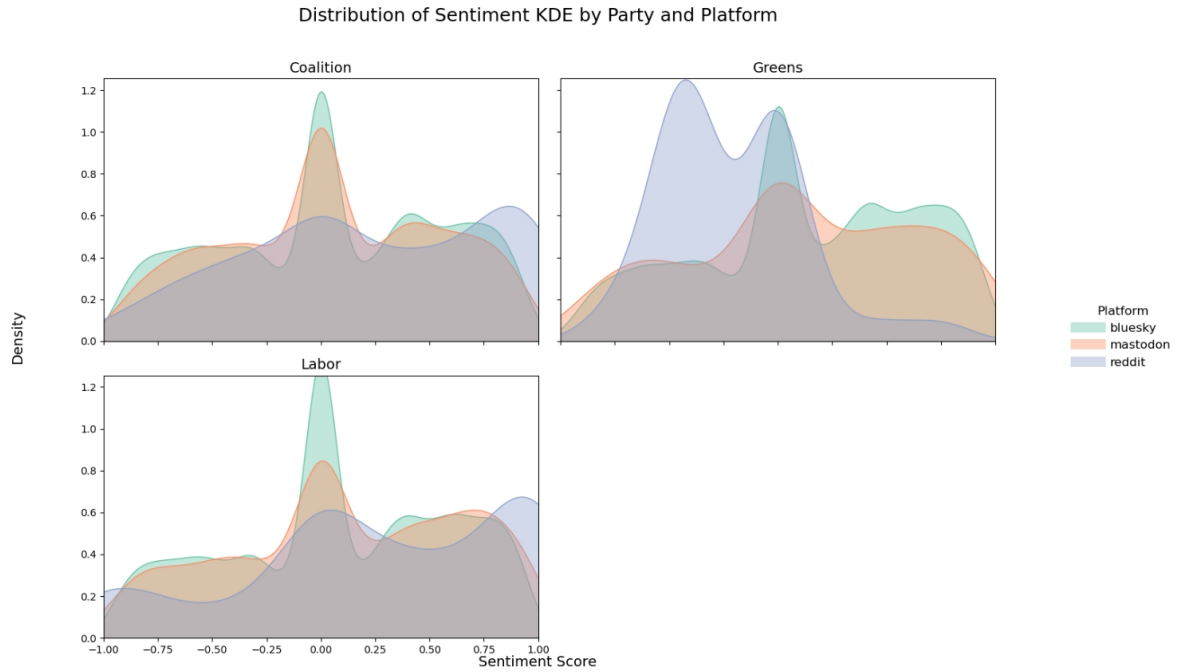

Figure 15: **Distribution of Sentiment KDE by Party and Platform**

The above three KDE graphs (Figure 15) show the differences in the emotional distribution of the three major parties on the Mastodon, BlueSky and Reddit platforms since the 2022 general election. Overall, Reddit has the greatest emotional fluctuations and contains more extreme voices. The discussion on BlueSky is relatively concentrated and leans towards the positive side. Mastodon is more prominent in terms of negative emotions. Specifically, Coalition is mainly slightly positive on all three platforms, but the negative aspects are more obvious on Reddit. Greens has gained more support on Reddit, while Mastodon has received significant negative reviews. Labor is distributed similarly on the three platforms, but the negative tail of Mastodon is longer. Overall, the ecological differences among various platforms do indeed influence the emotional direction of political party public opinion.

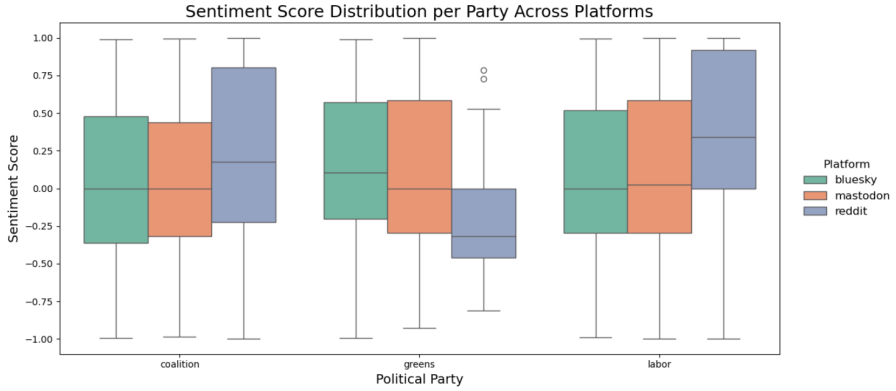

Figure 16: **Sentiment Score Distribution per Party Across Platforms**

The box chart (Figure 16) above reflects the differences in the emotional distribution of the three major political parties on the three platforms of Reddit, Bluesky and Mastodon. The median emotions of Reddit users towards both Coalition and Labor are positive. Especially for Labor, it reaches approximately +0.35, showing a stronger supportive attitude, accompanied by a larger fluctuation range. But the emotions towards Greens are more negative and concentrated. In contrast, the median emotions of Bluesky and Mastodon are mostly concentrated around 0, with a more compact and balanced distribution, indicating that the discussions on political parties are relatively restrained and less divided. Overall, Reddit is more likely to magnalize positive or negative attitudes, while Bluesky and Mastodon exhibit more rational emotional expression characteristics, highlighting the profound impact of different platform atmospheres on the expression of political emotions.

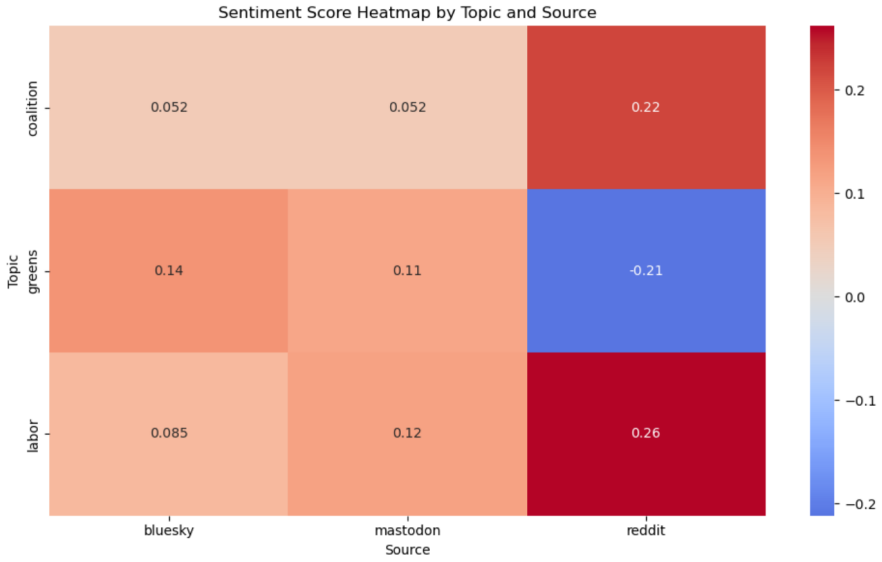

Figure 17: **Sentiment Score Heatmap by Topic and Source**

This heat map (Figure 17) shows the average emotional score differences of different political parties on the three platforms of Reddit, Bluesky and Mastodon since the 2022 general election. Overall, Reddit's emotional evaluations of Coalition and Labor were significantly positive (+0.22 and +0.26 respectively), while those of Greens were significantly negative (-0.21), demonstrating a stronger emotional amplification effect. In contrast, Bluesky and Mastodon had more moderate scores among the three parties, both remaining in a slightly positive range (+0.05 to +0.14), and their emotional expressions were relatively balanced. This difference indicates that the user structure and interaction atmosphere of different social platforms have a significant impact on the sentiment of political discussions.

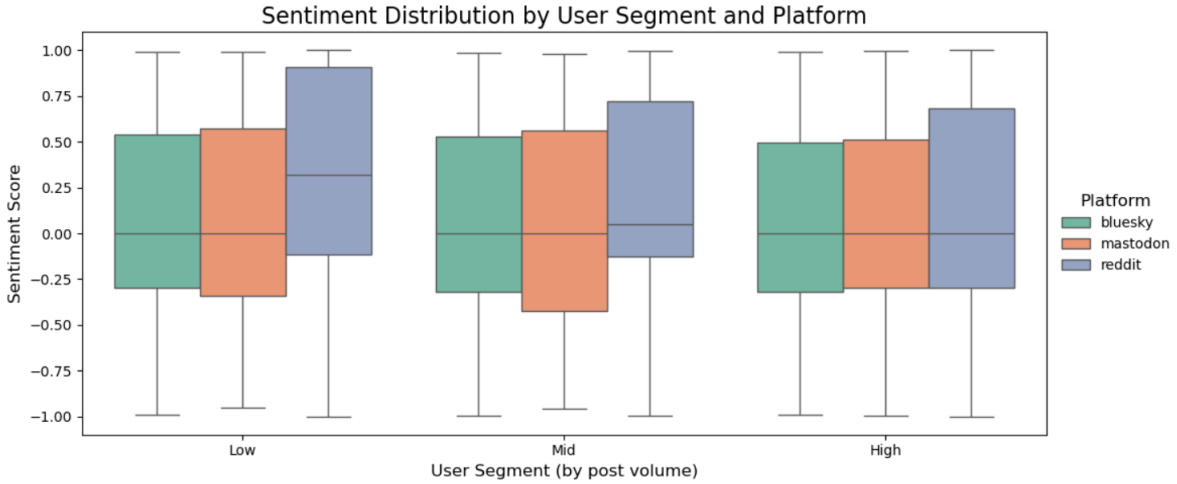

Figure 18: **Sentiment Distribution by User Segment and Platform**

This box plot (Figure 18) shows the emotional score distribution of users with different activity levels (Low, Mid, and High) on the three major platforms of Bluesky, Mastodon, and Reddit to examine the relationship between user engagement and emotional expression. No matter which platform it is, the emotional distribution of users with different levels of activity is relatively broad, covering from -1 to +1. However, the median and IQR show a consistent trend. However, on Reddit, whether it is low-frequency or high-frequency users, they tend to express themselves positively. While the median of Bluesky and Mastodon is usually around 0 or slightly lower, the emotions are more neutral or slightly negative, suggesting that the influence of the community culture of these platforms on users' emotional expression is relatively consistent. Furthermore, this graph also indicates that the influence of activity level on the emotional distribution is limited. High-frequency users do not exhibit more extreme emotions on any platform, suggesting that there is no significant correlation between the Posting frequency of users and their emotional tendencies.

Summary: Regarding Scenario 2, we conducted an analysis through multi-dimensional visual charts and found that the ecosystems of different social platforms significantly influenced the emotional expression of the Australian public towards the same political party. The Reddit community shows stronger and more volatile positive emotions towards Coalition and Labor, while the negative criticism towards Greens is also the most concentrated. In contrast, the discussion sentiment on BlueSky and Mastodon is more moderate, neutral and relatively consistent. Whether it is in the kernel density distribution, box plot quantification indicators, or the average sentiment heat map, it shows that these two platforms do not have too many extreme tendencies, but are mainly characterized by slight positive and negative fluctuations. Furthermore, the Posting frequency of users (high frequency/medium frequency/low frequency) has little influence on the emotional distribution, indicating that the platform attributes themselves can affect the differences in political emotions.

To sum up, since the 2022 Australian federal election, we have analyzed the public opinion's enthusiasm and emotional attitude towards the three major parties from two aspects: the "temporal and spatial dimension" (Scenario 1) and the "platform ecosystem dimen-

sion" (Scenario 2). Different times, regions and platforms jointly interweave the complex situation of political public opinion in Australia. Major events can trigger emotional fluctuations at specific time points. Political sentiments in different states also have their own emphases, and the platform community culture determines the emotional expression style of users on political party topics. Such multi-dimensional analysis not only helps us grasp the overall picture of public opinion, but also provides a solid foundation for subsequent refined research.

5 Error Handling and Fault Tolerance

5.1 Infrastructure-Level Challenges

Throughout the NeCTAR deployment phase, we encountered several practical challenges while stabilizing the core infrastructure.

(1) **Fission hook conflicts:** During repeated Helm installations of Fission or Kibana, pre-existing Kubernetes resources—such as `configmap` or `serviceaccount`—would often trigger `already exists` errors. These were mitigated by either explicitly deleting the conflicting objects or by isolating components into dedicated namespaces (e.g., `kibana-ns`).

(2) **Kibana startup delays:** The Kibana pod occasionally lingered in a `server not ready` state for over 10 minutes, due to memory allocation issues. We resolved this by increasing the memory limit from 512Mi to 1Gi and extending the container timeout to 300 seconds.

(3) **Port-forwarding limitations:** Due to restricted public routing in the NeCTAR environment, direct access to Kibana or ElasticSearch required SSH tunneling. Reliable fallback access was achieved using long-lived `kubectl port-forward` sessions for secure external exposure.

5.2 Harvester Fault Isolation

Given our decoupled architecture—where harvesters push to Redis and processors are triggered asynchronously—harvester-level failures did not propagate through the system. This isolation allowed us to absorb errors and delays without halting the pipeline.

(1) **Reddit integration** leveraged Pushshift.io's historical API to bypass Reddit's frequent rate limits. Although Pushshift had its latency quirks, it was more stable for batch ingestion.

(2) **BlueSky integration**, while still experimental due to incomplete public APIs, was implemented in a sandboxed manner. Partial failures (e.g., connection drops or malformed responses) were logged but safely ignored, ensuring the pipeline remained operational even with partial source coverage.

5.3 Content-Level Limits

Working with social media content inevitably surfaces issues of ambiguity, inconsistency, and noise—especially when applying lightweight enrichment methods. We encountered several limitations in semantics and natural language processing:

(1) **Sarcasm detection** proved problematic. Hashtags such as #DuttonForPM or #VoteGreen often carry an ironic or mocking tone. Our initial approach, which used direct keyword matching, lacked the nuance required to detect sarcasm, a limitation we acknowledged due to the real-time demands of our system.

(2) **Location extraction** was another challenge. Posts that referenced place names within larger expressions (e.g., "Canberra Times" or "Melbourne Cup") were sometimes incorrectly tagged as geolocations. We mitigated this by applying stricter regex filters that matched only standalone capitalised place names likely to refer to geographic locations.

(3) **Party inference** also required careful handling. Users frequently referenced multiple political parties in a single post or used abbreviations such as "ALP" or "Greens." To handle this, we employed a weighted scoring system that favoured dominant mentions and defaulted to a "`others`" category when ambiguity remained high.

5.4 ElasticSearch Indexing Exceptions

Indexing at scale is rarely flawless, and during bulk ingestion into ElasticSearch, we encountered several exceptions—mostly due to malformed JSON, missing fields, or schema mismatches.

(1) Bulk upload was implemented using the `helpers.bulk()` method from the `elasticsearch-py` library. To avoid cascading failures, each ingestion job was wrapped in try-except blocks, allowing us to isolate and skip problematic entries.

(2) For failed requests, a simple retry mechanism was used. While not overly complex, this ensured that transient ElasticSearch issues (e.g., overload or short-lived parsing failures) didn't result in permanent data loss.

(3) To aid debugging and future analysis, all rejected documents were redirected into a dead-letter NDJSON file named `rejected_posts.ndjson`. This enabled systematic inspection of failure cases and helped improve data quality iteratively.

5.5 Scalability and Fault Tolerance

To support real-time ingestion at scale and prevent data pipeline bottlenecks, our system was designed with horizontal scalability and fault isolation in mind.

(1) The Redis-backed Fission function is stateless and scales horizontally. Multiple instances can process events from the Redis queue in parallel, allowing the system to absorb spikes in harvesting or data throughput.

(2) Redis queue depth was continuously monitored using the `LLEN` command. While we did not deploy Prometheus in this iteration, the architecture allows for alert-based scaling using metrics such as queue backlog.

(3) Harvester re-initialization was managed using cron-based triggers. If a harvester encountered repeated failures (e.g., API timeout), the process would automatically restart

every 10 minutes, allowing for graceful recovery without operator intervention.

This architecture allows the system to scale with demand and continue processing even when individual components slow down or fail.

5.6 Logging and Observability

Logging and observability were critical for debugging and verifying the stability of our modular pipeline, particularly during active harvesting and function execution.

(1) Fission logs were accessed using `fission fn logs` and piped into custom log processors. These logs included timestamps, status codes, and enriched post summaries, which helped track malformed data and performance bottlenecks in the enrichment stage.

(2) For Redis, the queue length was periodically logged to stdout to track ingestion health and backlog patterns. This was helpful in identifying underperforming harvesters or overloaded processors.

(3) All harvesters wrote structured logs containing request timestamps, number of posts fetched, and encountered exceptions. These were essential for both performance profiling and error tracing, especially during rate-limited or throttled API interactions.

Although we did not deploy centralized log aggregation via ELK or Prometheus in this version, the system was built with modular logging hooks to support future observability upgrades.

6 Teamwork Management and Resources

This section provides a overview of team collaborations, including summary of contributions, challenges, and links to demonstrations.

6.1 Summary of Contributions

During our project, each team members are allocated with different jobs to make sure the smoothness of collaborations. The details of contributions are listed below:

- Angqi Meng: System design, implmentation of all Fission functions including harvester, processor.
- Yichen Long: Restful API, Implementation of tests, including unit test and end-to-end test; Sentiment abstraction and extract location; harvest and process of reddit.
- Xuan Wu: Set up of Elastic Search, data sampling and processing; report structuring and writing.
- Jingqiu Meng: Implementation of frontend, including data visualisation and analysis; frontend part, supported scenarios and analytical results report writing.
- Zining Zhang: Implementation of frontend, including data visualisation and analysis; report writing; Restful API.

6.2 Challenges

During the project, our team maintained effective communication. All members were open to discussions and willing to share ideas. Everyone actively contributed to the project, which helped us work efficiently and stay aligned throughout the development process.

However, a few challenges were rose during the project.

- The frontend was highly dependent on data provided by the backend, which created challenges for parallel development. To address this, we pre-harvested over ten thousand social media posts to use as sample data for visualisation. This allowed the frontend development to progress independently and be later integrated with real-time data once the backend components were completed.
- Our team members came from different academic backgrounds, which led to varying levels of knowledge in specific areas. Initially, randomly assigning tasks proved to be inefficient. To improve collaboration, we adjusted our task allocation based on each member's strengths and familiarity with the tools or concepts. This change significantly improved our efficiency and overall workflow.

6.3 External Links

The full source code is available at:

`https://gitlab.unimelb.edu.au/angqim/comp90024_team_60/-/tree/master`

The demonstration video is available at: https://youtu.be/KUbg1lIB0KY

7 Conclusion

7.1 Limitations

While the system delivered a functional and scalable election monitoring pipeline, several limitations emerged due to practical and technical constraints:

(1) Sentiment and stance detection were not fully integrated into the enrichment stage. Although the infrastructure supports sentiment metadata, we relied on a pretrained NLP model without fine-tuning, and did not implement stance-specific classifiers due to time and resource limitations.

(2) Topic assignment and location detection remains rule-based, relying on keyword pattern matching rather than more advanced machine learning techniques such as named entity recognition (NER) or transformer-based classification. As a result, party affiliation inference may suffer from reduced accuracy in edge cases.

(3) BlueSky integration was limited. With the restriction of Bluesky's API, the maxmium cursor we can take is limited on 10,000, indicating that we cannot track data back to three years ago.

(4) Reddit & Mastodon: With our current understanding of these two APIs, they only support live harvesting to get non-historical data, such restrictions limit our ability to mine deeper, failed our initial idea of performing a year-long analysis.

Although these constraints do not undermine the validity of our pipeline or its core findings, they highlight opportunities for deeper analysis and richer metadata extraction in future iterations.

7.2 Future Work

Based on the current system's foundation and observed limitations, several directions have been identified to enhance functionality, robustness, and analytical depth in future iterations:

(1) **Sentiment Enrichment**: Integrate fine-tuned sentiment analysis models (e.g., VADER, TextBlob, or transformer-based classifiers) into the processor stage to produce more nuanced sentiment labels. This would improve temporal trend analysis and emotional profiling of political discourse.

(2) **Topic Modeling and NER Integration**: Move beyond keyword rules by adopting machine learning approaches such as BERT-based NER or BERTopic. These techniques can uncover latent topic structures and named entities, improving topic classification and party mapping.

(3) **BlueSky Harvester Completion**: Resume BlueSky integration once API access stabilizes. This would round out the multi-platform coverage and improve representativeness in political content sampling.

(4) **Streaming Dashboards**: Implement real-time dashboard updates using technologies like WebSocket or Kafka. These integrations would allow Kibana and external dashboards to reflect pipeline data updates with minimal latency.

(5) **Automated CI/CD Pipelines**: Establish a GitLab-based CI/CD workflow to support container rebuilds, harvester updates, and system upgrades in a reproducible and efficient manner.

Collectively, these enhancements aim to evolve the system from a stable prototype into a production-grade, extensible analytics platform for political monitoring and beyond.

7.3 Conclusion

This project demonstrates how a well-architected, cloud-native infrastructure can be built to support real-time, multi-platform analysis of political discourse. By integrating Kubernetes, Redis, Fission, ElasticSearch, and Python-based enrichment logic, we developed a modular system capable of ingesting, processing, and visualising public social media data in response to emerging election narratives.

Our solution effectively bridges raw platform content and structured, queryable data — enabling researchers, policy analysts, and institutions to access timely insights on political engagement, sentiment trends, and geospatial attention patterns. While currently scoped to the Australian federal election context, the architecture is extensible and transferable to other use cases involving event-driven social media monitoring.

Ultimately, this work offers a reproducible and scalable blueprint for combining modern infrastructure with lightweight data enrichment to power next-generation social listening.

8 Appendix

Scenario	**Function**	**Type**
Post Count	countposts	query
Social Media Harvesting	mharvester bharvester-h rharvester	get data get data kafka queue
Data Structuring Pipeline	rprocessor mprocessor bprocessor	kafka queue kafka queue kafka queue
Query Ingestion Control	querysentiment enqueue	query query

Table 1: Function List

Scenario name	**Trigger**	**Category**

HTTP-based Data Query	`http://127.0.0.1:9090/posts/days/{date}`
	`http://127.0.0.1:9090/posts/days/{date}/topics/{topic}`
	`http://127.0.0.1:9090/enqueue/{topic}`
	`http://127.0.0.1:9090/querysentiment`
		httptrigger
		httptrigger
		httptrigger
		httptrigger

Scheduled Harvesters	@every 5s @every 40s @every 5m	timetrigger (bharvester-h) timetrigger (mharvester) timetrigger (rharvester)
Message Queue Integration	addobservations-mq bluesky-mq mastodon-mq reddit-mq	mqtrigger mqtrigger mqtrigger mqtrigger

Table 2: Trigger List

9 References

Helm Authors. (2024). *Helm documentation*. Retrieved from `https://helm.sh/docs/`. Accessed: 23-05-2024.

Kubernetes Authors. (2024). *Kubernetes documentation*. Retrieved from `https://kubernetes.io/docs/`. Accessed: 23-05-2024.

Redis Labs. (2024). *Redis documentation: In-memory data structure store*. Retrieved from `https://redis.io/docs`. Accessed: 23-05-2024.

ElasticSearch Authors. (2024). *Elasticsearch documentation*. Retrieved from `https://www.elastic.co/guide/en/elasticsearch/reference/index.html`. Accessed: 24-05-2024.

Kibana Authors. (2024). *Kibana documentation*. Retrieved from `https://www.elastic.co/guide/en/kibana/current/index.html`. Accessed: 24-05-2024.

Fission Authors. (2024). *Fission documentation*. Retrieved from `https://fission.io/docs/`. Accessed: 27-05-2024.

Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.